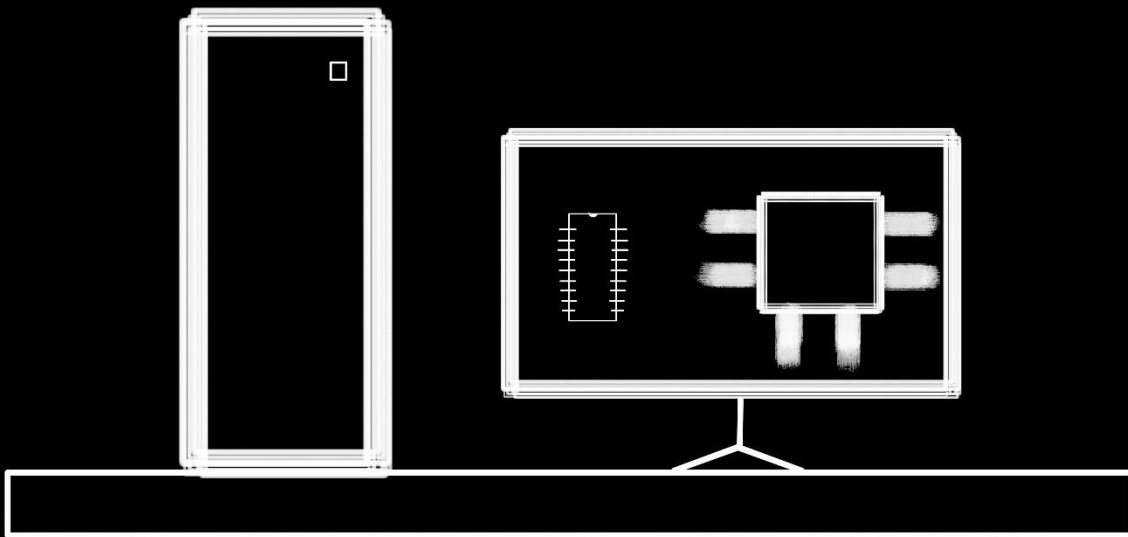


COMPUTERS

The Engineering and Programming Concerns,
Concepts, Approaches and Implementations



ADSkeptiK (on Github)

The page is unintentionally left empty

The Structure of this book

The book is organized in subjects, chapters and sections. Each subject is to cover an extremely generic and vast CS subject that is the groundwork of many generic roles/jobs. We then have chapters which with the exception of chapter 0s specifically and in detail focus on job paths and projects for specific hardware used in industries, to get the user actually job ready. In chapter 0s we focus on getting the general concepts right and making the user ready for starting to then focus on the more specific routes. Then there are the sections for noticeable chunks of lessons that are related to a specific aspect of the chapters focus. Then there are Lessons which are taught in each section. For example: Chapter 0 of Networks focuses on the general computer networks stuff while the next chapter focuses on Cisco's specific routers or CCNA or stuff like that. This E-book was digitally written and uploaded by ADSkeptiK (Github) for free and is expected to be shared for free in it's digital form at least.

ADSkeptiK



Subject 1: Computers & Programming

Chapter 0: The General Challenges and tools

Introduction

Before computers there was programming! The combination of knowledge in the form of noticing repeating patterns and creativity out of the love for saving time and increasing efficiency had our ancestors programming and automating chores since long ago. Time saving in spite of being the starting concern, became ONE of THE important concerns in programming. With the gradual growth of the societies, civilizations and the domain of knowledge, the domain of possible applicable solutions expanded as well and alongside them the concerns over resource management, flexibility and etc also emerged. The realization of the importance of

management resulted in understanding the important concept of “**System**” and the birth of this very concept as a logical solution. As our systemic and management thinking grew we started to replace humans within those systems with tools to save time and for systems to be safe of certain limitations that human resources would bring with them. It was these ambitions that ultimately led to the invention of **Computers**, tools for saving and doing logical computations and eventually mathematical calculations.

Computations and Logic

In Theory

Computations which are the main point of computers, fundamentally have 2 undeniable and necessary requirements to work with:

- 1- The actual data that we want to use for computations (the Knowledge)
- 2- An algorithm/pattern/thinking patterns to compute those data with (The Creativity)

Since we live in a world that ultimately finds itself under the influence of the unknown, we could only prepare and ready ourselves to increase our chance of success for as much as our domain of **knowledge** and strength allows. Under these circumstances naturally:

- for data: the **preciseness** and **exact-ness** become the main but not the only ideals.
- for Patterns: **the higher chance of success ends up on the side of the one that is more ready for both a wider range of possibilities AND has the ability to handle them well,**

In order to get close to the ideals for these 2 foundations of computations, having **accurate definitions** becomes a very important factor in both. Accurate definitions are given by **accurately checking the standards** that are met (A standard is a complex condition based on a group of definitions that are based on units. A unit is a small simple measuring system defined in relativism to a constant). The best way to accurately check the standards (or to define them even) is to use smaller informative binary conditions that are either Met or Not Met, to decrease the ambiguity.

In Computers

The **condition based nature of computations** demands for computers to also be ready for **conditions and possibilities** to be able to fulfill their main purpose of existence. In order to achieve that and get, set and describe the data and patterns mentioned in the previous part **computers tend to use digits as they could be used for possession measurement , counting, calculations, Identification (describing data AND possibilities)**. We'll get more into what I exactly mean here in the first section.

The levels to programming

There are considerations for starting programming and then there are also layers to programming, from soldering literal hardware pieces to writing code into files on computers with the goal of implementing routines could all technically be regarded as programming. Regardless of which layer you wanna program in, one thing's clear: You need knowledge of the other parts that you'll need to use and deal with in that layer and you also need to come up with efficient plans to achieve the best

out of what you have. We'll get to the general flow of programming and the important concepts in each layer for now and get to the details for implementing actual projects in each layer later. It's safe to say we're beginning from the very bottom and slowly going towards the top in the computer levels. The model below is a perfect way to represent what we're doing.

(Place Le Model pic here)

Section 1: The hardware (Electronics)

In this layer you deal with micro architectures, soldering and etc.

Before we can get comfortable I'm gonna need to first re explain some common knowledge in a more powerful way that warms up your mind ready to understand things a lot easier from this point on.

Unique Symbol Set

A Unique Symbol Set (UNSS) is **a finite set of symbols** the members of which represent states established by the UNSS's specification specification.

E.g: UNSS_Decimal = {0,1,2,3,4,5,6,7,8,9}

Here the symbol 1 means a single instance of a thing, 2 means a single instance added by another and... . Keep in mind **a finite set of symbols does not necessarily mean a finite set of combinations and permutations of them.**

Digit

A digit is a limited finite container that could fit just a single member of a **UNSS**.

E.g: $D \in \text{UNSS_Decimal}$

Here D is a digit, it could only contain a single member of UNSS_Decimal set at a time.

Digit Organization

Conventions

The **Digit Organization Conventions (DIOC)** define how multi digit values are supposed to be organized and interpreted to give the context meant according to their purpose. A DIOC is a hypothetical ideal (usually unavailable) manual that tells us how to read and interpret digits, we tend to build this manual in our head by gathering chunks of information about the UNSS and the digit organization as the two subjects are usually intertwined.

The Standard Counting Convention

The Standard Counting Convention or TSCC is just one of the common conventions that we organize digits with. It's the most commonly used DIOC in real life for dealing with numbers but not necessarily the only one or the end to them. TSCC as the name suggests evolved alongside humans as a way to make possession measurement easier. The main idea behind it is simple: We count based and in relativism to a established threshold or **numeric base** for all digits.

Naturally we'll need:

- 1- A single decided **numeric base/unit threshold** for all digits.
- 2- A limited UNSS containing symbols for representing all amounts to be used for describing and measuring instances (except for the amount of a fulfilled threshold). Typically in UNSS members there's a symbol for the **lowest amount of instances (LAOI)** AKA nothing (0 in humanity's case) and a symbol for the **just-below-threshold amount of instances (JBTAOI)** (usually 9 in humanity's case). We intentionally do not define a symbol for the threshold itself cause the "symbol" of it and the counts above it are to be implemented using digital logic and relativism to the base.
- 3- Each time the amount of instances held by a digit exceeds/fills/reaches the threshold or threshold sized units could be extracted out of it, they are extracted as additions known as "**carry**" s to its left digit, these carries are according to the number of times the threshold sized units gets extracted.
Any remaining amount of instances remains within digit.

The floating point convention

Each of the possessing states could possess just as many states within itself each of which could contain just as many and it goes on.

Digital Logic

As mentioned before (*in 1.0.Introduction*) we use numbers in the TSCC convention in 4 general ways:

- **Possession Measurement**: To determine/measure the amount we own/possess of something.(Measurement has 0 in it cause we could "have nothing")

- **Counting:** To determine how many of 1 existing object we have.
- **Calculations:** To keep track of changes.
- **Identification:** A number could be used to represent a unique possibility.

Digital logic is about at times using all these capabilities of numbers at times in parallel to their full extent to achieve and assist computations.

Numeric Systems

The number of different possibilities/states a single digit can represent is set by it's numeric system. A **numeric system defines UNSS and the numeric base**. Since in computers at the basic level we are really using numbers to measure the amount of data that the components “possess” or “own” we could also say the **Numeric Systems in CS set the threshold of the states of possession/count of unique possibilities that a digit could hold (which we'll call range count from this point on)** (yeah I know you probably understood nothing, but it was just an extremely short and beautiful way to sum it up, it sounded so good).

- For **Decimal** numbers it's 10 (You could possess from 0 to 9 states for each digit (**10** is the range count / number unique possibilities per digit)).
- For **Octal** numbers it's 8 (You could possess from 0 to 7 states for each digit (**8** is the range count / number unique possibilities per digit)).
- For **Hexadecimal** it's 16 (You could possess from 0 to 15 states for each digit (**16** is the range count / number of unique possibilities per digit)).
(Notice the word Count instead of value. It plays a big role later)

NOTE: Range count does NOT conceptually equal Base and it is possible to have and define numeric systems that have different range count and bases. They are

usually the same because we start from zero in our common possession measurement usage of numbers (which is what we'll be dealing with 99% of the times) but just to be sure the generic formula for calculating it is:

JBTAOI – LAOI or Initial USS Symbol

The formula calculates the actual distance of the initial symbol in UNSS till it reaches right before the base (or till it reaches the **JBTAOI** itself), giving you the actual certain .

Bit

For computers to compute:

- 1- They need to keep track of data, they do so using numbers (the 1st generic way of using numbers *as mentioned*).
- 2- As we said previously (*in 1.0.Introduction*) **binary conditions** are common in computations as they allow for (kinda) terminating ambiguity.

As a result of the two statements above the **binary numeric system** became a perfect candidate cause a **binary digit** AKA **bit** could be used in the places of any of those binary conditions to hold their state hence bit is the smallest basic unit of measuring the “vastness”, length and size of the possibilities in computers and since the said sizes in computers are usually at least in 10s of bits and above, Byte was adopted as a more common bit-based unit for the same purpose, each byte equals 8 bits. Aside from Byte the physics SI prefixes have also been totally repurposed in the field of Computers and CS to be used for bits AND bytes. In CS:

- Kilo = 2^{10}
- Mega = 2^{20}
- Giga = 2^{30}

- Tera = 2^{40}

All these repurposed CS SI prefixes work for both Bits and Bytes. That said the problem of ambiguity still very much remains in CS as believe it or not sometimes people use the physics definition of the prefixes for CS as well. Some newer prefixes such as kibi for kilo and a like have been defined to hopefully exterminate the confusion but haven't become all that dominant and popular yet hence it's good to actually read the manual of apps to see which is the one being used.

Numeric System Conversion

Digit Equalization

In order to convert numeric systems from numeric system A to numeric system B we need to first determine how many digits of numeric system B equals to a single digit of numeric system A. This is achieved thru using $\log_B A$ where A is the range count of numeric system A and B is the range count of numeric system B.

Value Equalization

Our handling of the value conversion from one numeric system to the other is greatly dependent on 2 aspects:

- 1- Whether the target numeric base is smaller or bigger than the initial one.
- 2- Whether we are dealing with fractional or whole values.

Whole numeric value conversion

When we're dealing with a whole number If:

- 1- Target numeric base is bigger than the initial one:

- 1- We sensibly divide the value to the target numeric system's range count.
 - 2- The remainder of each division becomes the value of the digit.
 - 3- The resulted dividened is the amount that has made it passed the digit and goes thru step 1 to 3 again till the number left cannot be divided.
- 2- When we're dealing with a smaller

Complements

In complements regardless of diminished or non diminished we swap them to the complement necessary to get them to their **in range threshold** .

Data Types

Due to the *mentioned reasons* the threshold of the size/length of the data we can contain is always finite and it's representation done bit based. According to the size of our data we deal with at the moment we use different sizes of containers from the finite range we have to hold our values accordingly.

The way numeric data saved in a datatype are to be read and saved (their encoding) is and must be different based on whether they're fractional or integral. The integral values :

- 1- Signed and unsigned integers
- 2- Signed and unsigned floating point numbers

Signed and Unsigned floating point

[sign][Mantissa]e[Exponent]

The number of bits for mantissa and exponent are fixed.

Vaccum Tubes

Transistors

Buses, Words

*Buses are collections of electrical conduits that carry bytes of info
between components. These fixed-size byte chunks are known as
words.* **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

I/O

The I/O s typically connect to I/O bus using a controller or adapter.

COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Controller

Are chips in a device itself or parts of the motherboard.

COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Adapter

A card that plugs into a slot on the motherboard.

COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

ISA

*Processors operates based on a simple instruction set architecture
AKA (ISA).*

Micro Processors

CPUs basic operations are always in one of these groups:

*Load: Copy byte/word from memory to CPU register, overwriting
previous val.*

Store: Copy byte/word from register to memory.

*Operate: Copy contents of two register to ALU to perform an
arithmetic operation and then store the result in a register
overwriting it's previous value.*

Jump: Extract word from the instruction itself and copy it into program counter. Overwriting previous val. **COMPUTER SYSTEMS**

A PROGRAMMER'S PERSPECTIVE

They execute machine instructions they get from a memory. They have a word size storage device (register) called program counter (PC) pointing at the address of machine instruction in their memory. The processor reads instruction from pc and interprets and does the operations. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Registers

Register File

Register file is a small collection of word size registers, each with its own unique name. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Cache

They serve as temporary staging areas for the information likely to be needed in near future. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

L1 Cache

L1 Cache on processor chip holds tens of thousands of bytes and can be accessed nearly as fast as the register file.

L2 Cache

L2 Cache holds hundreds of thousands to millions of bytes and is connected to the processor by a special bus. Accessing it is 5 times slower than L1 but still 5 to 10 times faster than memory. L1 and L2 caches use a hardware technology called Static Random Access Memory or SRAM for their implementation.

ALU

The ALU computes new data and address values

Micro Computers

Micro Architecture

Defines the actual implementation of the processor and it's ISA

.COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Endianness

This is one of those subjects that is usually stupidly explained... so now I explain it well so you wouldn't have to go thru the same shit I did to finally understand it

It's used to indicate the load direction of the sequence of input bytes that fulfill our n-byted buffers from their highest bit downward in a CPU architecture :

1- The loading of these input bytes could begin from the first byte in front of them to forward (Big Endian)

OR

2- the nth memory byte given to them backward (Little Endian)

Main Memory

A temporary storage device that holds both a program and the data it manipulates while the processor is executing the program, it physically consists of a collection of dynamic random access memory (DRAM) chips.

**COMPUTER SYSTEMS A PROGRAMMER'S
PERSPECTIVE**

Section 2: The Operating Systems (OS in particular)

Flow of Apps in modern systems

The machine instructions for an app are originally stored on disk.

When program is loaded, they are copied to main memory, as the processor runs the program, instructions are copied from main

memory into the processor. **COMPUTER SYSTEMS A**

PROGRAMMER'S PERSPECTIVE

The processor fetches an instruction from memory, decodes it (i.e., figures out which instruction this is), and executes it. Thus we have just described the basics of the Von Neumann model of computing.

OPERATING SYSTEMS THREE EASY PIECES

Virtualization

OS's typically handle the efficient interaction, management and timing between components easier thru the use of an engineering concept known as **virtualization**. “Virtualization” abstracts the pain of hardware specific physical layer to make them look and appear as usable blocks to the user.

APIs

APIs allow using the “virtualized” hardware.

History

System V is a unix standard BSD is another.

OSs

(System's Engineering, Systems Programming, System's Design Level) In this layer you deal with computer organization. Firmwares, BIOS and etc.

Memory

Logically memory is organized as a linear array of bytes, each with its own unique address (array index) starting at zero. **COMPUTER**

SYSTEMS A PROGRAMMER'S PERSPECTIVE

Programs use different sizes of memory and to be able to have control over the bytes of memory, they NEED to have unique identifiers for each of those bytes. With data representation in computers ultimately happening by bits, the same unit is used to state the length of the those unique identifiers for memory bytes in computers, the length is referred to as **address space** and has direct effect on how much memory you can use at the same time.

Memory Hierarchy

The main idea behind a memory hierarchy is that storage at one level serves as a cache for storage at the next lower level.

COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Direct Memory Access

DMA

Virtual Memory Address (VMA)

Why it's a thing: The programs can't know for certain in which memory addresses they're gonna be loaded until their runtime but until then they still gotta rely on some independent in house addressing system to do their relocations and all and that's how and why they use the virtual memory address space which is a middle man addressing system keeping the program connected until it's actually loaded at runtime. Now the number of unique memory addresses or to be more frank, the size of memory that can be accessed at the same time, is define by how many bits long the address space can be.

It's safe to say virtual memory is an abstracted interface to access dynamic random access memory (DRAM), flash memory, disk storage and special hardware.

A machine level program views memory as a very large array of bytes referred to as virtual memory. Each byte of which is represented by a unique number, called a virtual address. The set of virtual addresses is referred to as virtual address space. **COMPUTER**

SYSTEMS A PROGRAMMER'S PERSPECTIVE

Memory Page

Why is it even a thing: To manage apps that areNow it is neither convenient nor optimized really to load apps at their entirety, hell the memory of the computer may NOT EVEN BE ENOUGH to hold them entirely. Each of these chunks have a contiguous group of addresses is called a **page**. The pages of an app form what's **virtual address space**. For loading the app the wrapper unit for determining how much **virtual** memory is used is **page** (which itself is based on bytes obviously) and each page is a contiguous series of bytes. For physical memory the same kinda wrapper unit exists and is called **pageframe**. The pages and page frames typically are of the same size.

System Call

Is a special function that passes control to the operating system.

COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Context Switches are managed thru system calls to kernel for managing process creation.

Kernel

Kernel is part of the operating system that always resides in memory. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Kernel is not a separate process but a collection of code and data structures that system uses to manage all processes. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

CPUs

A central processing unit is typically made of one or more processing cores, cores each capable of handling one sequence of instructions at a time, on their own.

Process

A process is the operating systems abstraction for a running program. Multiple processes can run concurrently on the same system, and each process appears to have exclusive use of the hardware.

COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Context Switching

Traditional systems could only execute one program at a time, while newer multi core processors can execute several programs simultaneously. In either case, a single CPU can appear to execute multiple processes concurrently by having the processor switch among them. The operating system performs this interleaving with a mechanism known as context switching. When the operating system decides to transfer control from the current process to some new process, it performs a context switch by saving the context of the

current process, restoring the context of the new process, and then passing control to the new process. The new process picks up exactly where it left off. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Now obviously the number of tasks AKA sequences of instructions we are running may exceed the number of cores which is why these cores use a technique known as context switching to “Jump” on another

Threads

A CPU/Processing core executes lines/threads of instructions by reading them thru it's own threads. Each processor and application can consist of many threads executing within it's address space, which the processor's threads could lock onto. The threshold of the number of threads that a processor can simultaneously execute is naturally limited to the number of threads it has.

Multi-Threading

In case a CPU's total threads are lower than the number of threads in a process, context switching becomes the go-to solution. Otherwise CPU threads each get assigned to a process thread and handle them simultaneously

In case of hyperthreading, each core technically achieves the illusion of handling multiple threads thru its

The virtual memory of a process contains different sections for code, data, heap , shared libs, stack and kernel virtual memory.

Multi-Threading

A process can actually consist of multiple execution units, called threads, each running in the context of the process "11-d sharing the same code and global data'.

Amdahls law

Principle: The performance and time efficiency is directly based on the increase in the efficiency of components that contribute to it and the domain of said component in the whole system.

Formula in simple words: The new time performance of system equals to the sum of the previous time performance of the unchanged components and the reduced time of the improved component.

$$T_{\text{new}} = (1-a) T_{\text{old}} + (aT_{\text{old}})/k$$

$$= T_{\text{old}} [(1-a)+a/k]$$

In the formula:

- a is the domain of component.

- 1 is used to refer to the entirety of system to later be used for calculating the time of the parts outside the domain of component.

- k is the factor of performance, since here we're calculating the reduced time NOT the performance boost to it we divide instead of multiplying.

And then the formula for speedup becomes $S = T_{\text{old}}/T_{\text{new}}$

$$S = 1 / ((1-a) + a/k).$$

$$0.2 + 0.8/k$$

Section 3: Programming

Languages

Common Concepts

Floating Point numbers

In case of using the scientific notation the mantissa's evaluation could be modified using parentheses.

Rounding Up Numbers

The formula: $((\text{sizeof}(n) + \text{sizeof}(\text{int}) - 1) \& \sim(\text{sizeof}(\text{int}) - 1))$ is pretty much a well used familiar structure for rounding numbers within Microsoft's compiler definitions.

Common logical errors

Off by one error

Since in loops we simultaneously use a number for identifying, counting and calculating. Things tend to get super confusing and more often than not the number of counts end up being more than what we had in mind. In order to overcome this obstacle remember these:

- 1) In loop counter we merely set the starting point of the count.
- 2) When the loop begins you are always already in the first count.
- 3) Considering your loop threshold is in positive numbers and it goes thru incrementing. The formula $(1 - (\text{counter starting point}))$, marks the offset of the actual count of your loops from the desired.

Interpreters

Compilers

The C Programming Language

C was introduced and is used as a compiled programming language... it's its whole premise. Just like any other programming language, knowing the syntax of this language is one thing and the way we tend to program in it in jobs is another and the ready to use library it has is another, hence we start by getting straight to the programming approaches in this language, followed by explanation for the basic everywhere stuff such as syntax and its keywords and finally the library.

General flow of C programs

Application Building Process

Programming in C tends to be made of the following steps:

- 1- Source Code Modularization: The C source code is modularized by putting the definitions of related functions, structs and global variables in files with descriptive names for the side of program that they implement, followed by .c extension, this way each C file becomes a module of our code.
- 2- For each module (.c file) we create a header file with the same name as the module but followed by a .h extension instead, in which we place the declarations for all the definitions in the module.
- 3- The .c files are then given to C compiler as input. The compiler first processes the Preprocessor directives (parts of C code).
- 4- The preprocessed files are each checked to make sure the declaration of each function, structure, variable is present in the file **BEFORE** its place of usage in code. The passed files (called translation units in this stage) are then

turned into their respective machine code files (AKA object files usually with the .o or .obj extension). The (the single) entry point of the program is found and “marked”.

- 5- The linker resolves the calls to functions in other object files, links/stiches them all up together, looks for the (the single) entry point of the program and sets it as the beginning of the code and turns them all into an executable binary, a program that could be run by the OS or a static or dynamic library (both of which we discuss in the section).

CRT (C Run Time) What is it

CRT is a block of code that’s added before the entry point and is responsible for calculating and “saving” command line arguments and possibly assigning the main’s parameters to them according to the standards (such as the case with argc and argv).

The Programming Process

Libraries/APIs

These are essentially groups of functions, structures and etc written by other programmers that we use in our applications. These libraries come in 2 forms.

- 1- Static Libraries: Compiled object files of function definitions to be linked at linking stage.
- 2- Dynamic Libraries: Compiled files to be accessed at runtime.

Regardless of which kind the library is , the code of the program using them obviously needs the declaration/prototype of the functions defined within them for compiler not to nag in pre linking stages. For that reason we always need the “include” header of these libraries that’s including just the very same prototypes.

It’s in the very same header that we implement the conditional inclusion and conventions to make sure only the right declarations are loaded for the functions of the library and are therefore “accessible” throughout the code.

The Entry Point

Generally the default entry point of C programs is to be their **main** function (although certain APIs could and DO demand a different function names with different and varying numbers and types of arguments as their entry, (an example being Win API) which is exactly why you’d check the documentations regarding the implementations of C in said APIs before jumping into programming but for now we’ll stick with the general API independent main entry that’s expected to work on most systems), main is to return int and either have no parameters (in which case the parameter should explicitly be written void!) or have parameters int argc and char*argv for systems with command line environment support for C programs. The reason for this is that the process of running the executables of C programs begins by first setting the command line arguments (argv and argc) thru an initializer program called C Run Time that then calls the main function. The first argument in systems with command line environment support for C programs is assigned with the number of commands given as input to the standard input stream in the stack “before hitting ENTER” and the second argument is assigned to holding the string array that contains those arguments.

Build Specification

Before clicking on Build the configuration (debug or release) for build alongside the architecture and project dependencies (possible reliance on other projects to be built) is set and then the building begins.

Modularization and cross platform-ability

Approaches

Modularization and cross platform-ability approaches could take place in 4 stages:

- 1- Preprocessor Stage (Macros)*
- 2- Source Code Stage (Encapsulation)*
- 3- Compilation Stage (Translation units)*
- 4- Build Stage (libs and Usage of smaller exe s thru Command lines)*

Preprocessor Level

The modularization in this level decides the **inclusion of the right chunks of code according to system's parameters**. Cross-platform-ability of codebases is typically implemented within this level. Some very known methods used in this level are:

- 1- ***Label Conditionalization***: Compilers and libraries of code are typically written to conditionalize a list of system parameters as labels for usage in ***conditional inclusion***, the definition of these labels (using the #define LABEL or compiler settings) later leads to the inclusion of certain corresponding code chunks after preprocessing. 2 best examples for this are the _UNICODE and UNICODE labels included in MSVC C/C++ empty

projects by default within the Preprocessor settings of the projects.

`_CRT_SECURE_NO_WARNINGS` is another example.

- 2- ***Conditional Symbolic Function-Overload Abstraction***: (It's a self made long name but a descriptive one for a common method which I'll refer to as **CSFOA** from this point on... It's still more descriptive than shit like Virtual Private Server!): There are times where a systemically generic function would have multiple system-parameter-dependent versions (such as the generic systemic functionality of reading character which could have it's 1 byte character and wide character versions or a GUI function that should incorporate different infrastructure on Linux than it does on Windows), it's in these times that we use method 1 to define the name of the systemically generic function to be replaced by the name of the right "overload" version of it.
- 3- Name convention implementation: Naturally the possibility of name conflicts in C is generally higher due to the absence of some really helpful features such as classes, function overloads and custom namespaces that were added in C++. With all the said options out of the window using macros for adding prefixes and postfixes to function names is the way that's been used to enforce naming conventions on them in C.

Header Files

Header files are used to include the declarations for functions and types and keeping track of them till they are resolved. Putting the info that's likely to change

Source code level

Functions

Compilation Level

Translation Units

At this level modularization happens thru the use of .c files. The definition of functions and structs that depend on similar infrastructure or help in fulfilling a single generic goal (usually both) are placed together in a single .c file (right after the inclusion of the declarations for their required infrastructure within the very same .c file of course). Naturally the ideal for efficiency in this level is to try to avoid including unwanted declarations by our .c files. We can get closer to this ideal from 2 aspects:

- 1- By getting the most out of each header inclusion: Through placing functions that use the same infrastructure module we assure that we get the most out of the included header in .c file.
- 2- Using Optimized headers: Include a custom made header that ONLY contains the needed declarations.
- 3- Manual Forward declaration: Selectively and manually write only the declarations for the part of infrastructure module that's actually used in our module, on top of the .c file.

Ultimately since in C each .c file becomes a translation unit it's best to make sure code is distributed among .c files in a way that changing a mere part of the program wouldn't need or demand changing or recompiling its entirety but the translation unit we have changed.

Header Only Libs

Sometimes folks may put the entire definition of functions in headers instead of just a declaration. Such approach could be regarded as bad practice for when functions are likely to change, as it shows little flexibility for changing functions in future, logically demanding the recompilation of all translation units that have included the header in one way or another for any and every change.

Link Level

Usage of Static Libs

Static libs are archives of **object code** s

Usage of Dynamic Libs

Build Level

Static Libraries (.lib)

They are binaries following a structure that linker can use for relocating the references or connecting them to DLLs.

While linking, these libraries either have one header that includes the declaration for all the functions within them, or the number of nested headers that do so next to the static lib file.

WARNING: It's very important for the static lib to have a simple name. Avoid bullshit, spaces and ' and odd characters as much as possible as I personally ran to a configuration parameter error related to "Additional Dependencies" setting in MSVC/C++ while trying to link my static lib once.

There's little difference between machine code object files and static libs but the difference is noticeable nevertheless more specifically in windows. Lib files are archives of those machine codes and translation units in single respective libs.

Regardless of the format of static libs, the logical structure and accessibility to them is a matter of importance. In order to access a library the client of it needs the declarations of the functions or at least the declaration of the functions they need to access. While from outside the cleanest least painful approach would be to have the declarations for all the contents of all the translation units in the lib, written in a single header to then include and use them; the same "ONE BIG HEADER with ALL the declarations" approach doesn't look all that good while developing the library, and it only shows itself significantly when the size of the actual header gets big. It's lack of modularization.

Which is why I've personally come to prefer the header modularization approach for development and then the single big header for accessing library approach. Although again depending on situation these approaches could get different. Such as the case with Windows API for instance. So yeah it's a whole other level of engineering on it's own as well, often not described at all in computer engineering or CS courses for some reason (Yeah there goes CS for you).

Dynamic Link Libraries (.dll)

The compiler behavior

Default status of const values

The C compiler tends to see values as constant signed by default and any time it's not stated to be unsigned.

Read Only Data

Values such as string literals are saved in the read only section of the application and regarded as const. Any attempts to change the rodata value by the app becomes an access violation.

Static/Dynamic Memory Allocation

The Language

Data Types

All data types are ultimately broken down to bits. With data types we merely define the number of bits and how they are to be treated.

char

A typically 1 byte data type that can hold upto 1 byte of values signed and unsigned representation of values is determined by the keywords and by default it's signed. It can be assigned with both variables and literals of It's own kind and the integer kind so long as the integer is within the 1 byte range.

Character Constant Literal

These literals are one byte signed integer literals represented between single quotes. A character, Octal digits and Hexadecimal digits could be used to indicate the value that's meant by them.

Using a single character would indicate the ASCII value of the character:

e.g : `char c = 'D'` //Means take the ASCII value of D and give it to the variable.

Pointers

Pointer Casting

In order to cast pointers use the following notation:

`(casttype *) pointername;`

Operators

Operators are used for managing the use of data. They have an order of precedence to which we must pay attention while using them to get our desired effects.

Binary Operators

Used to mask off bits (for shit like rounding up numbers).

Ternary Operator

? operator

This operator is used for conditional existence of a piece of code where we can't use if. It kinda behaves like a conditional inclusion preprocessor whose life time continues after the preprocessing stage.

Bitwise Operators

! Operator

It's sensitive to zero and turns it to 1 otherwise turns it to 0.

Definitions

Chain Assignment

We can perform chain assignment in the following form:

```
int var_a=0, *ptr=NULL, var_b=-1;
```

Declarations

Control Statements

for loop

The iteration variable always ends up at the very condition. That's why unless you're careful, you define it inside the parenthesis.

2 things to keep in mind regarding for loops:

- 1- The loop counter SHOULD and consequently does end at the fail condition. Which is why it's advised to define counter within the parentheses as otherwise you MIGHT forget this and use the fail value-ed counter unless you're aware of what you're doing.
- 2- The loop counter pre-counts at the beginning of the loop, this means that if the condition breaks in the middle of the current cycle, the cycle is still counted. This could cause trouble if you don't pay attention.

My ultimate idiom for you to keep in mind for getting the loops right for me personally is: "Make sure the count only begins after the passing of the first object".

Functions

C functions don't allow overloads.

Headers

Preprocessor Directives

These directives affect the source code in the source files before compilation.

They could be one line or multiple lines in order to force them to consider the next line use a backslash \

Note: There shouldn't be anything other than a newline character/enter followed after the backslash, even a space after it would cause error.

#include

Is one of the many directives used for including header files. Literally copies and pastes the files in front of it in its location. It could be used in 2 ways:

- `#include <filename.h>`: Searches the compiler's include directories for the header.
- `#include "filename.h"`: Searches relatively to the address of the source code it's defined in.

#define

Is used for defining symbolic constants and macros

#if

#ifdef

#ifndef

#endif

Static

Static keyword can be used for functions and variables and would cause a different behavior for each:

Static Functions

By defining functions as static, their definition only becomes ONLY visible to the file that they are defined in(meaning even if you include their declaration in other files they could not be linked to any definition, resulting in undefined link error or the use of another available definition with the same name).

Static Variables

Defining variables as static could cause 2 different behaviors in them depending on the scope their scope:

- For global variables: Defining them as static makes them only visible to the file they're defined in.
- For local variables: Defining them causes them to keep their value since the last time their function was invoked.

Apendix A: Data Types

integer

Apendix B: Functions

Apendix C: Headers

Apendix D: Preprocessors

Apendix E: Scopes and Storage

The Standard Library

Headers

stdio.h

Contains the typedefs for many types thru the inclusion of other headers which at the end rely on OS specific headers. In Microsoft's Windows this reliance is on vcruntime.h which is indirectly involved and included in many if not all headers.

stdarg.h

va_start()

Is a function/macro used in functions with variable length argument lists, responsible for setting it's first argument to point at the first unnamed argument of

the function, using the second argument given to it which should be the identifier for the last named argument in function parameters.

va_end()

Cleans up the function variable length argument list.

function

fgetc()

The flexible version of fchar capable of reading characters from streams (files) other than the standard inputstream(also known as stdin) which was the file that getchar() read from.

fputc()

The flexible version of putchar capable of writing characters to streams (files) other than the standard outputstream.

feof()

Checks to see if the EOF marker for the given argument of it has been reached.

scanf()

The function takes a list of conversion specifiers. After parsing each, it decides whether to see the bytes as an ASCII byte. It takes as much length as the first argument has defined and then any extra values will be given to the following args.

Data Type	scanf conversion specification
-----------	--------------------------------

double	%lf
--------	-----

long double	%Lf
-------------	-----

float	%f
-------	----

unsigned	%llu
----------	------

long long int	
---------------	--

long long int	%lld
---------------	------

unsigned	
----------	--

long	%lu
------	-----

int	
-----	--

long int	%ld
----------	-----

unsigned	%u
----------	----

int	
-----	--

int	%d
-----	----

unsigned	
----------	--

short	%hu
-------	-----

short %hd

char %c

FILE

Is a file structure containing the info that program needs to process the file. It may or may not contain a file descriptor, an index to an array in OS containing additional settings on handling file.

stderr

standard error stream.

stdin

standard input stream , can be used with putchar and getchar.

stdout

standard output stream.

fgetc

The function **READS** a character from the file given to it as argument in the form of a pointer

fputc()

The function **WRITES** the character in its first argument to the file specified by the FILE pointer in its second argument.

fopen()

The function is used to create a file with the name(extension included) of it's first argument for the mode defined in it's second argument. It returns the address of it as a FILE pointer.

feof()

The feof function's argument is a FILE pointer to the file to test for the end-of-file indicator—stdin in this case. The function returns a nonzero (true) value when the end-of-file indicator has been set; otherwise, the function returns zero (false). This program's while statement continues executing until the user enters the end-of-file indicator.

fprintf()

Writes into the stream given to it's first argumnent.

sprintf()

Print f but prefixed with a new first arg that acts as an output buffer for printf.

malloc()

Takes the number of bytes to allocate and returns a void pointer to the chunk of array of the number of bytes it's allocated. It is okay with the number being a variable. We typically mix it with the sizeof function and a pointer type casting so that the memory is as much as needed and accessed accordingly as well:

```
userdefinedtype* userdefinedPtr= (userdefinedtypename*)  
malloc(sizeof(userdefinedtypename*n);
```

Error Reading

The function is declared but not defined

Causes: Possible use of static function in a file other than the module.

The argument in additional dependencies was supposed to be Boolean

Possible spaced or 'ed naming while defining the dependencies.

The C++ Programming Language

C++ is C with extra capabilities and headaches as well. Read the C language before this so that I can only focus on the C++ specific stuff here.

Classes

Class's Type Definition

Class definition in C++ happens in the following format:

```
class classname{};
```

Class's Member Function Definition

Member functions of a class are defined in to ways:

- 1- Fully defined the within the Type Definition of the class with the following format:

```
class classname
{
returnType memberfunctionName(type param1,...)
{body of the function}
};
```

- 2- Prototyped in Type Definition and later defined in modules: In this approach With the assumption that the type definition is presented and has the prototype of the member functions within it, we can use the :: scope resolution operator to define the member functions in detail in the following format:

```
class classname
{
function1 prototype;
function2 prototype;
};
```

```
Return type for prototype 1 classname::function1(...) {}
```

Class's Object Instantiation

the instantiation can but does not always happen after the class definition:

```
class classname {} instancename;
```

More often than not the instantiation happens like a normal variable definition
(Assuming the class's declaration and definition have been seen linked):

```
classname instancename;
```

Member Functions

Constructors

A constructor is an essential role among functions that one member function must ultimately play. The way the constructor is decided varies depending on which of the main 2 circumstances we are dealing with:

- 1- No candidates for constructor : In this scenario there are no suggested functions for the role, hence the compiler automatically generates a default empty constructor at compilation.
- 2- There's a list of candidates for constructor: In this scenario the user has defined a constructor or overloads of constructors for different circumstances. In this scenario the compiler will either force you to meet the required circumstances for at least one of the overloads of the constructor and then call it OR if the required circumstances are met it jumps to calling the constructor that fits the situation.

Constructor Delegation

There are times when the thing that happens in one overload of the constructor also happens at the beginning of another. In such times constructor delegation is one good approach for code reusability. In short: Constructor delegation is when a constructor overload calls another constructor overload to handle the identical first step present in it with given arguments, instead of completely redefining that first step. It's achieved using the following format:

```
constructorName(params): constructorname{params for the  
constructor that runs first} {The of the constructor to  
run after the overload was run with those parameters}
```

Preprocessors

extern “C” {C function declarations}

Makes C++ compiler load C function declarations accordingly.

__cplusplus

If the compiler is a C++ compiler.

The Java Programming Language

General flow of Java programs

After the installation of Java runtime and all... and setting the PATH variables so that java compiler and java virtual machine become accessible from the commandline, we could use:

javac filename.java

to compile our java file to bytecode and then use

java filename

to run the now byte code version of the program on JVM.

Java sees every program as a bunch of classes, member functions (called methods) and their variables. Naturally as a result of this, the concept of **Global Scope**

Functions simply does not exist in Java, as all functions are **member functions/methods**.

By default Java imports the *java.lang* package which is *rt.jar* in its *Java/JDK/jre/lib* directory.

The content of java API's libs */rt.jar*s within IDEs is usually shown as actual source code which is in contrast with how JAR files contain the bytecode, this is because for convenience Java IDE's (at least in the case of IntelliJ) view a different archive, the source code archive for the existing byte codes, which is typically in the root folder of Java and is named *src.zip*.

In short the main difference between C/C++ and java is that Java looks at source code like a bunch of already existing modules to connect are not instead of pieces that may or may not conditionally get copied into the code or not. Preprocessing doesn't much exist in Java.

Projects and package organization

Every java project must settle a directory as it's root. The files that are **just within** (NOT the subfolders or files within the subfolders) will not belong to any packages hence should have a settled package declaration to mark them being in the root. The files within subfolders within the root however always have to declare their path in relativism to the root.

Java Modularization

Java treats each source files as a “representative” (and home) of a public class for convenience. Naturally as a result this means each source file would get 1 public class.

1- Preprocessor Level (Macros)

2- Source Code Level (Encapsulation)

3- Compilation Level (Translation units)

4- Link Level

5- Build Level (libs and Usage of smaller exe s thru Command lines)

Preprocessor Level

Source Code Level

Class

It's pretty much like in C++ except this time the access to classes can be limited. Since in Java modularization on all levels happens and gets enforced thru based classes, it'll affect other levels as well, hence, it'll show up in the latter levels.

Getters and Setters

In IntelliJ: We can use alt+Enter while on the variable line to define getter and setter for the variable

We can also use Alt+insert to add getter and setter for variables.

Abstract Class

Interface

Compilation Level

Class

In Java each a source file is only allowed to be as the placeholder of a main class with the same name inside it. Other than that there will be a compilation error.

These main classes could be defined in 3/4 ways:

- Public: In which case ALL other modules could access them.
- Private: In which case only the modules within the same package could access them.
- Protected:

Packages

Packages are mere directories containing java classes. The C files defined in them must reference themselves as parts of the package using a **package declaration** at their very beginning, using relative path.

Link Level

Packages

In Java a group of classes form this concept known as packages.

JAR files

A jar file is a ZIP archive (Yes just like the same kinda zip archive you normally see in games/etc) that contains byte-code/.class files of libraries.

The Programming Process

The Entry Point

Naturally there must be an entry point for the program this entry point will be in the form of a static function (So that there wouldn't need to be an instantiation from it's class to be accessible) named main. There should be only ONE function with this name that the JVM binds to while loading .class files.

```
static void main (string[] args){}
```

is the natural entry point of java programs.

Tip

arrayname.fori is a good shortcut to create loop for array.

Local Variables

In Java local variables are not initialized by default.

The Language

Preprocessor

Import

Used to include definitions of classes.

Operators

+

Aside from its already existing mathematical functionality that it had in C/C++, the operator allows for “seamless” concatenation of numbers/non character data into strings.

=

The assignment operator.

Comments

// and /* are the goto keywords for comments with the exact same meaning they had in C/C++.

With the addition of /** which is a more specific and preferred commenting format that causes JDK to use them in creating an HTML documentation.

Data Types

Primitive data types are written in small form while the non primitive types are written with the first letter capital.

Use camel case for variables and method and pascal case for classes

Basic Data Types

Instantiation

Derived Data Types

Classes

Classes are user defined types that could and usually do include a bunch of variable/ constants and some behaviors.

Unlike in C++ where classes themselves did not have access modifiers and were always globally available, in Java the availability of the classes is defined by their access modifier and there wont be a semicolon after the definition. Each class definition follows the following format:

AccessModifier *class* **className** {}

Due to the intertwined nature of classes and files in aside from what was already said any public classes must be declared in a file with the same name.

Instance Variables

Unlike in C++ instance variables do not use the colon format but are simply prefixed to the instance variables, per variable.

The format is :

AccessModifier **Type** **name;**

Conventionally we put the instance variables on the top of the class so that it could be

Note: Instance variables ARE initialized by a default initial value.

Methods

In Java methods are defined once in the java class with the access modifiers written per function. The rest of the format is identical to the one in C/C++:

AccessModifier returnType className (1stParamType 1stParamName, 2nd ParamType 2nd ParamName).

Note: Local variables are NOT auto initialized by default.

Method Overload

In order for methods to be overloaded we have to have atleast an extra parameter in the similar method or a parameter with a different type or sequence. The return type does not matter.

Method Override

Overriding== We have a method that takes exactly the same arguments(calls the first method in it's body) + extra things in it's own body. Overriding methods can be achieved by using the method prototype and then defining the new body in it's overriding method in a subclass. The return type name and parameters have to be the same in overriding. We also can't have more limited access in the overriding methods only less limited access modifiers are allowed as our overriding methods require at least the same level of freedom as the overridden method. Static Methods can't be overridden.

Final Methods

Final type methods cannot be overridden.

Constructors

A constructor is a method with the name of its class that makes sure the variables that are in its parameter list are initialized.

this

The keyword **this** has the exact meaning as it did in C++. Since in Java we don't have pointers, the **this** means the dereferenced pointer to the object not the mere pointer.

Class's Instantiation

In Java the instantiation of classes is an unusually extra stepped declaration in comparison to C++ as instead of using the format:

Type name ;

That was used in C++ and for Java's normal variables, you have to deliberately assign the named declaration to an object of it using keyword *new*:

Type name = new Type() ;

Subclass

Subclasses are classes that are inherited from the other one. Constructors can also be inherited by subclasses but their name sensibly requires a change. That's where a super constructor call comes in handy. A super call basically calls the parent constructor within the constructor of the child class.

Abstract Class

An abstract class contains prototypes for functions to be defined in the subclasses. When trying to create an object of an abstract class the abstract methods must be defined right after the constructor in a block as well. we use implements to inherit abstract classes

Interface

Interfaces are just abstract classes that are only prototypes. In interfaces access modifiers are public by default. Interface reasonably can't inherit (extend) a class but only extends an interface

Generic Methods

Used when types are different but the method's action is the same. First we have to define a type parameter like this. right after name

Generic Methods

Instantiation: `ClassName objname= new classname<>();`

Expressions

The API / Standard Library

The Java's API is usually (in IDEs like IntelliJ at least) visible under the External Libraries section in solution explorer section of the IDE. It is implemented as a set of JAR files (ZIP format Archives) containing a set of directories and the metadata required for unarchiving them correctly. As a result these JAR files could contain from "folders" to files/public class containers.

With the IDE's usually importing Java API by default as they also set the directory of their residence set an include directory, there ain't much need for importing them.

By default (IntelliJ IDE at least) has the root of import set at the **rt.jar**.

String (class)

System (class)

Contains stuff... related to System. Prominently used for I/O operations.

E.g: `System.out.println("String")`.

Contains:

- an object of type `PrintStream` named **out** thru composition.
- an object of type `InputStream` named **in** thru composition.

PrintStream (class)

Provide options for printing. Some known methods involve:

- `print()`
- `println()`

- printf()

Scanner (class)

Defined within `rt.jar.java.util.Scanner` it is used as a place holder object that could be tuned to accept inputs. Contains the following methods:

- `Scanner(filestream)`: Constructor used to set where to read the input from.
- `nextInt()`: Reads another int from keyboard.
- `nextLine()`: Reads characters until it reaches newline/Enter character which it discards and replaces with `\0`.

The x86 Assembly

The XML

The Python Programming Language

Environment & Tools

Industry hack: The python code is used in small chunks and run thru bash/batch files indirectly for checking and debugging different parts of the apps.

File Management

Dynamic Libraries

CRT is a block of code that's added before the main function and is responsible for calculating and "saving" command line arguments and possibly assigning the main's parameters by them according to the standards(such as the case with argc and argv).

Chapter 1: Developing an OS

"To write an OS for a Von Neumann computer, a programmer needs to be able to understand and write code that controls the core components: CPU, memory, I/O devices, and bus." __Operating Systems from 0 to 1

Chapter 2: Windows Programming (WINAPI)

Environment & Tools

In order to develop applications using this API having/downloading the Microsoft Windows development tools (headers, compiler and all) is necessary. Windows greatly depends on 2 main Dynamic Libraries; one of which includes Kernel mode side of the functions with the other one including the User mode side functions to achieve many of it's tasks. With the User mode dynamic library containing the functionality and being the most bare minimum abstraction that the Win programmers usually need, the header containing the function declarations of it are packed alongside the static reference libs, Microsoft's visual C/C++ compiler, a text editor, debugger and many much helpful necessary tools to be given as visual studio and it's windows SDK to the users.

Approaches

“Back-end”

Window Class Registry

The USER32 subsystem has a class table in which it holds the template of class for later-reuse in an efficient manner.

C/C++

C#

“Front End”

Entry Point

The WINMAIN gets the base address (the address at which the executable is loaded into memory) as hInstance at runtime, next the single string of command line argument string (NOT like in standard C's argv which is an array of strings that C runtime has already taken care of by separating arguments based on

whitespace by default) is received thru cmd as szCmdLine and the default state in which process is run based on OS's assessments is received iCmdShow.

Windows

A Window is a generic structure. In order to create one we first create an instance of this structure. Then we MUST configure it according to our ideal so that when we're done we can register it for the backend side.

Windows programs typically rely on communicating with "Window" objects using concepts known as messages. These messages are handled by a role known as a "window procedure", a function the parameters of which describe the particular message that is being sent by Windows and received by program.

Every Window has an associated window procedure which could be in the program or even a dynamic lib.

The Message Queue

For all windows programs a queue is made, holding all the messages to all the windows a program might create

Window Pump / Window Message Loop

In order to read and manage the messages of the queue according to the window of the application, Window pump/Message loop s are developed.

C/C++

Internationalization

Developing International Software for Windows 95 and Windows NT by Nadine Kano.

The Usermode DLL

Before the 32 bit era there was the 16 bit era in which the DLL for user mode side functions was a file called USER.EXE (Odd right but seriously it functioned as a DLL back then). It wasn't until the 32 bit versions of Windows showed up that the User

The implementation of much of Windows's types and MSVC compiler's behavior is typically achieved through:

- 1- **The CSFOA method:** You see, considering how much of Windows was first written in C before C++ and function overloading was a thing programmers had no option BUT to always define the “technically” overload versions of a function with different names and to give the illusion of overloading the solution was to use conditional preprocessor
- 2- **SAL:** Windows API uses a MACRO convention style known as SAL for it's implementation.

SAL

Is a descriptive language for defining macros used by Microsoft Windows almost everywhere for the implementation of Windows API and is thoroughly covered in sal.h header that comes as a part of the SDK

AFX Resource

Language/Script

Running Executables with parameters

By dragging files onto executables the address of them is given to executables(the directory of them/file.extension is given to the executables as parameter).

Resource Files

Resource files are files deployed alongside applications that fill gaps such as icons and etc.

.ico

These files typically contain a PNG and a couple of bitmaps

WinAPI specifics

Constants

CS_HREDRAW

Sets `wndclass.style` for redraw upon every change in height.

CS_VREDRAW

Sets `wndclass.style` for redraw upon every change in width.

Preprocessors

_Unicode

Defined in `UNICODE`, it's used to include the Unicode version of `tchar` functions.

TEXT(quote)

Turns string literal into the ASCII/Unicode syntax for initialization using the `__TEXT(quote)` Preprocessor.

__TEXT(quote)

It's a conditional preprocessor loading the string literal quotes according to the syntax.

DECLARE_HANDLE(NAME)

Used for the creation of handle types, defines NAME as a “pointer to a NAME_struct” type with a single int member.

Entry Point

The entry point of the average WinAPI program instead of the good old:

```
int main (int argc, char*args[]) { /*program*/ }
```

in C/C++ is the *WinMain* function with fixed parameters that tend be defined using the naming convention of one of the two prototypes below:

```
int WINAPI WinMain (HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nShowCmd)
```

```
int WINAPI WinMain (HINSTANCE hInstance, HINSTANCE hPrevInstance, PSTR szCmdLine, int iShowCmd)
```

where function's return type is *int WINAPI* (*WINAPI* being a convention for identifying the return type of WinAPI functions, it is mostly used in them).

- The *first-parameter* (*hInstance* here) is an instance handle, the unique number that identifies the program.

- The *second-parameter* (*hPrevInstance* here) was used to determine if other instances of program were running and now its abandoned in 32 bit Windows and is always defined as NULL in modern Windows.
- The *third-parameter* (*szCmdLine* here) is the command line used to run the program (some apps use to load a file into memory when the program is started), the **sz** here stands for zero terminated string.
- The *fourth-parameter* (*iShowCmd* here) is used to set whether the program should initially fill the window or be minimized or whatnot the **i** stands for integer in the name.

Functions

WinMain()

The prototype is typically one of the lower:

int WINAPI WinMain (HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nShowCmd)

int WINAPI WinMain (HINSTANCE hInstance, HINSTANCE hPrevInstance, PSTR szCmdLine, int iShowCmd)

MessageBox()

A CSFOA used to show short messages, it shows a dialogbox and it ain't much customizable. The prototype is:

int WINAPI MessageBox (HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption, UINT uType);

- The *first-parameter* (*hWnd* here) is supposed to get the handle of the possible “owner” or “parent” of it (**OR NULL if an owner/parent doesn't exist**) to process the message box relative to it.
- The *second-parameter* (*lpText* here) is supposed to be the text of the **body**.

- The *third-parameter* (*lpCaption* here) is supposed to be the text of **title** of MessageBox.
- The *fourth-parameter* (*uType* here) is a constant (possibly a bitwise combination of multiple constants) for setting flags to add some predefined and limited icons and options to the window. Using NULL or 0 here will give the default window with the default buttons, the default icon and the default selected-by-default button.

The full list of the constants will be written in Appendix A.

The message box tends to return different values based on buttons present varying from IDYES, IDNO, IDCANCEL, IDABORT, IDRETRY, IDIGNORE to 0 and 1.

When using this function we tend to put the string literal parts within a TEXT macro which is optional but helps in case we wanted to be ready to convert our programs to Unicode character set.

For now we use build ctrl + F7 to build project and then ctrl + F5 to run it.

wcslen()

Is used to determine the length of wchar array minus the '\0' character and is defined in both string.h and wchar.h.

wprintf()

Defined in both standard C's stdio.h, AND wchar.h.

strlen()

CSFOA used to calculate the length of string minus the terminating '\0' char.

strcpy()

CSFOA used to copy the second string into the first.

strncpy()

CSFOA used to copy the first 3rd parameter's number of elements to the first argument.

strcat()

CSFOA used to concatenate the first argument with the second one.

strcmp()

CSFOA Compares two string char by char if all are equal(in size and value, meaning strings are totally identical) returns 0.

strncmp()

CSFOA does strcmp but for the number of characters given in the third parameter.

sprintf() (C stdlib)

```
int sprintf ( char *szBuffer, const char *szFormat, ...);
```

The function is used to hold formatted string in a buffer.

vsprintf()

A variation of sprintf, where the first two arguments are char buffer and format string but the third is actually an array of arguments instead of being

GetSystemMetrics()

A useful function for obtaining info about the sizes of various objects in windows.

LoadIcon()

Used to load an icon for use by a program. First argument is the hinstance of EXE in which to look for and the second one is the ID of the

LoadCursor()

Used to load a mouse cursor for use by a program.

GetStockObject()

Gets a graphic object.

RegisterClass()

Actually registers class within the user32 subsystem as a template (read **Window Class Registry**).

CreateWindow()

Creates a window based on a window class.

ShowWindow()

Sets the window for showing.

UpdateWindow ()

Renders the window according to the latest changes.

GetMessage()

Obtains a message from the message queue.

UpdateWindow ()

Directs the window to paint itself.

GetMessage()

A message from the message queue.

TranslateMessage()

Translates some keyboard messages.

DispatchMessage()

Sends a message to a window procedure.

PlaySound()

Plays a sound file.

BeginPaint()

Initiates the beginning of window painting.

Headers

vcruntime.h

Much of the fundamental typedefs and build dependent size determination for implementing C and C++ is done in this header file using macros.

wchar.h

Contains the type definitions for wchar_t data type and wide char related functions such as wcslen.

string.h

Contains string related stuff and functions such as strlen.

windef.h

Includes type definitions and certain functions for them.

tchar.h

Acts as a cross-encoding supporting (ASCII & UNICODE) header abstraction for functions.

winnt.h

This bad boy handles the basic Unicode support

Types

wchar_t (C stdlib)

2byte charcters for representing Unicode. Literals of this type are to be prefixed by L (without any space in between) to indicate that each character in the literal is to represent 2 bytes.

TCHAR (Windows

Generic/Universal Char winnt.h)

Is a generic character type used determined at compile time based on whether `_UNICODE` has been specified or not.

CHAR (Windows 8bit char winnt.h)

Window's middle type name for 8 bit character.

WCHAR(Windows 16bit char winnt.h)

Window's middle type name for 16 bit character.

PWCHAR, LPWCH, PWCH, NWPSTR,LPWSTR, PWSTR (Windows defined in winnt.h)

All typedefs for pointer to WCHAR (WCHAR*).

LPCWCH, PCWCH, LPCWSTR, PCWSTR (Windows winnt.h)

All typedefs for pointer to Const WCHAR.

PTCHAR

Pointer to generic/universal char.

LPTCH, PTCH, PTSTR, LPTSTR

Pointer to the generic char type.

LPCTSTR

Pointer to constant generic char type.

WNDCLASS

A type used to keep the contents of window?

HWND

A handle?

MSG

Message Pump

It's a strategy to keep the window loop in the rendering mode.

NULL

Is defined in windows vcruntime.h.

Chapter 3: Visual Studio IDE stuff

Preprocessors

The default preprocessor inclusions `_UNICODE` and `UNICODE` are inherited by IDE by default.

Debugger

Most C/C++ IDEs have a debugger that allows adding breakpoints within software code for checking instruction and variable's values in a Local or watch section. Within the Watch section the domain of what you see is usually vast, in the local section you see the local function vars...usually only when in the debug configuration tho, while some control is still available in the release configuration in order to be able to observe each function step by step and see the local vars you NEED to be in debug config.

Endianness

Naturally the Debuggers tend to abstract away the endianness, showing the representation of the values AFTER applying the endianness.

Chapter 4: OpenGL Graphics Programming

OpenGL is a graphics programming API/standard offering a basic pipeline and a set of basic tools for achieving graphics programming.

Chapter 5: Binary Analysis in Linux

Section 1: Formats

ELF Format

The binary file format is used for both executable (.out) and relocatable (.o) object files generated by the C compilers (at least gcc). Either way they include some basic sections, remarkably a .text section for all the codes and a .data section for ...well identification data I reckon. While in the non stripped version things seem rather clear...At least relatively,in the non stripped version at least for now the best way to find and identify functions seems to be reliance on the address space they had pre stripping.

.init section

For initializing the program stub and the like.

.text section

It contains the machine code of the program.In the non-stripped output tends to contain all the functions and their names in the <functionname> format.

`_start` function seems to set the command line arguments.

`Endbr64` mysteriously marks the beginning of the functions and alongside the beginning address of the functions in non stripped version, could be used to find them in the stripped version.

DWARF Format

Another format used for debug information. It's usually embedded within ELF files but could be separate as well.

Section 2: The Loading Routine

Loading Programs

When a program is run, the OS makes a Process for it and assigns a virtual address space and an interpreter to that process.

Section 3: Tools

Text Reader

It just prints the bytes as the encoding that it supports.

cat filename.ext

It prints the bytes as ASCII characters

.

Dump Readers

These apps are used to specifically read the machine code sections (show the actual bytes) of a format and usually disassemble them and give VALUES.

objdump

It's a dump reader used to show the actual bytes.

objdump -s filename.ext

Used to show the bytes of the different sections of the file in text editor-ish manner.

objdump -sj sectionname filename.ext

Used to show the bytes of the specific section of the file in text editor-ish manner.

objdump -d filename.ext

Used to disassemble the .text section of the file.

Format Readers

These apps basically just read the file with the intent of showing the stuff such as symbols and general informative, locational stuff in a Format. Mostly to give information alone.

readelf

It's an elf reader. It's commands are used to load and give the attributes of ELF file formats (mainly it's data section).

readelf -a filename.ext

Used to load ALL the info about the elf.

readelf --syms filename.ext

Used to load the info about the symbols section of the elf file.

readelf--relocs filename.ext

Used to load relocation info about the ELF file.

Chapter 6: Binary Analysis in Windows

Chapter 7:

Internationalization of programs

Chapter 8: Android Development

Textures

Texture Compression

BC7

Texture mapping

My basic analysis

Witcher 2 executable

Chapter 6: Data

Compression

Data compression has to do with the methods and algorithms to believe it or not compress data (serious shit!).

Good examples of the kinda historical methods people came up with to achieve it are Braille, morse codes and vcoder (voice recorders) where:

- 1- (in morse code) Assigning less demanding forms to more frequently used alphabets (which was later used in Hoffman encoding as well)
- 2- (in braille grade II) Using a special symbol to determine whether the following content were to be read as word or alphabet, taking advantage of less symbols to cover more frequent words, leading to 20% compression.
- 3- (in Vcoder) the data related to frequencies that could not be heard by man were just avoided.

Lossy/Lossless

Compression

Subject 2: Computer Networks

Chapter 0: The General Flow

Introduction

The communication between numbers of computer devices with each other for sending and receiving data in the form of packets lead to the formation of computer networks (Which are protocol-ed communions between computers). The connection of computer networks to other computer networks and the exponential extension of the network lead to the formation of the **internet/interconnected network** as we know it (which is an extremely large protocol-ed

communion/gathering of computers).The communion itself is made possible thanks to the implementations of certain general and purpose specific key concepts in the form of hardware AND software infrastructure that exist to specifically handle the process of taking and posting packets from and to the computers.

Section 1: The Basics

Lesson 1: Internet from An Abstracted View

Key Roles

Now for the formation of a computer network there are certain logical roles that would be necessary to fulfill different yet equally necessary aspects of the job. For easier understanding consider Internet as a massive network of Post-offices for computers that it is. There are the Writer and Target Recipient devices for packets that are both referred to as **End Systems or Hosts** (Didn't use the terms sender/receiver cause many devices might technically "send" and "receive" the

packet in the process until it reaches the desired destination) and then there are the “post office” devices which receive and dispatch the packets to their destinations using their links and are called **Packet Switches**. The end systems and packet switches communicate and send packets through their **socket interfaces** and follow certain **protocols** so that the **packets** (yes the data that computers send in networks are literally called packets here and are sent in similar fashion as well !) are sent to the right destination based on a list of allowed routes.

Conclusion

The whole functionality of internet ultimately really relies on 5 logical basic concepts:

- Packets: Structured data (based on protocols/standards) ready to be sent to a single or a group of end systems.
- Protocols: The structure rules for packets, requests and etc and how they are meant to be sent.
- End Systems: The systems that post/receive packets. Servers, Phones, Computers, etc.
- Socket Interfaces: The hardware/software infrastructure responsible for sending/receiving the packets.
- Packet Switches: The devices that provide route/link to other End Systems.

Lesson 2: Connection to Internet

Packet Switches' Goal Division

For our devices to “connect to” or “access the internet”, they must contact the packet switches that allow routes to the “target recipient” of their packets (otherwise they could not be delivered), as everytime we use internet we’re sending some request to some device and then (usually) getting some request from that very device in response. Divided by purpose, we have 2 main groups of packet switches that are made to serve different yet equally necessary parts of the process of packet delivery:

- Link Layer Switches: They form a district of all the devices that are connected to them by providing them all with usable links that the connected devices can always use for sending/receiving packets to and from all other devices that are connected to the Link Layer Switch. Going with the previous post office example it’s safe to say they act as the “neighborhood” local post office.

(Note: The main concern of the link layer switch is not “whether to provide a link or not” but “how to provide the link”)

- Routers: They regulate the transfer of packets using their gates to make sure only the packets that meet the priorities and rules defined by the operator makes it. They are conditional connectors. (THEY ARE THE DISPATCHER).

Lesson 3: Access Networks

Knowing the purpose of Routers and Link Layer Switches, if we try to model internet we'd see how both the End Systems and the first router of a network always end up on the sides/ends/edges of the network. End systems of a network logically and as the name suggests ... "end" up being on the end/edge of the network as they are its limits and then its first router (being the connector of the said network to the other ones that it is), also ends up on the edge of it (Used the word end a little too many times there I know Lol but it's to fully install the idea in your head). That's why access to a first router (Usually called the **Edge Router**) in order to access the internet becomes an important and deciding factor. The infrastructure/kind of network that exists to connect separate end systems/computer networks to the edge router is called an **Access Network** (In other words an access network is:
a computer network/end systems + the infrastructure that connects it to its first router). Since the subject is one of those parts that could easily become extremely confusing. For simplicity and better understanding how access networks work, I use the following ring observation and approach to teach what is actually happening:

- 1- Ring 0: The Pre-Edge Router Network/ End Systems
- 2- Ring 1: Linking to the Routers

1. Ring 0: The Pre Edge Router

Network/ End Systems

This is the inner-most ring we always start from. “The computer network/The end systems?! The hell is that supposed to even mean” you see in this ring we ultimately have one of the 2 scenarios mentioned below (And they are NOT THE SAME):

A- Separate End Systems : Just a bunch of end systems **without connection to each other.**

B- A Computer Network (LAN): A group/pack of end systems **that are actually connected to one another as well.**

In scenario A at the ring 0 we only have the End Systems and NOTHING more, we ourselves do not “have” our own computer network. In scenario B however we already have a small computer network of our own (also referred to as a local area network or LAN) which necessitates the addition of a link layer switch and it’s linking infrastructure to the ring to form that “pack” of devices(So scenario B is conceptually: Scenario A with links in between).

The linking to the link layer switch happens in one of the ways mentioned below:

- 1- Wireless: In this kind the link layer switch itself or some connectable infrastructure to it provides what’s called a base station (AKA Access Point or AP for short) that could be used by supporting devices for wireless connection.
- 2- Wired: In this kind of connection, Ethernet cables are used to connect devices to the link layer switch.

2. Ring 1: Linking to the Routers

Even the edge routers could exist at extremely varying distances and the scenario we have in ring 0 is likely to indirectly affect that. For . If it's scenario A we are dealing we are dealing with. Just like with the link layer switches, there are 2 general groups of common ways to link to the edge router, that are used under different circumstances:

- 1- Wireless: The connection to the routers happens through a satellite like with starlink/hughesnet and etc or using an access point (just like the case with link layer switches).
- 2- Wired: The connection happens using physical connection. It's still more common.

For wired connections depending on how far the router is engineers came up with some damn good strategies on how to connect to the routers which as I'm writing this have remained the famous and mainstream ways so far. These 4 common types of wired links to the router from a connection infrastructure perspective are:

- 1- DSL: It's common in home access. In this kind of edge router link local teleco transfers data to the edge router and vice versa using the existing Telephone lines (not coaxial cables). This happens by grouping the data on the phone lines using frequency ranges (One range for two-way telephone channel, one for uploading/sending data and one for downloading/getting data from internet). On the customer side, a splitter separates the data and telephone signals arriving to the home and forwards the data signal to the DSL modem (modulator/demodulator). On the telco side, in the CO (Local

central office), a DSLAM(Digital Subscriber Line Access Multiplexer) separates the data and phone signals and sends the data to the Edge Router.

2- Cable Internet (HFC): It's common in home access. The cable television company transfers data to the edge routers using a combination of fiber cables and coaxial cables since the two kind of cables are used, this kind of access is also known as Hybrid Fiber Coaxial or HFC as well. In this kind of access network data transference to and from houses happens using coaxial cables that then reach a main coaxial trunk. The coaxial trunk is then connected to a fiber node. The fiber node is connected to the cable television company's HQ using fiber cables. Inside the cable television company's HQ there exists a cable modem termination system (CMTS) that serves a conceptually similar purpose to DSLAM and does the process of turning analog signals taken from the cables back to digital ones. Instead of DSL Modems special cable modems provided by the cable television company are used. It's a shared broadcast medium.

3- FTTH: It's common in home access. There are two architectures for its implementation known as Active Optical Networks (AON) and Passive Optical Networks (PON). We'll discuss PON for now. In PON, The teleco's CO offers the infrastructure in the form of fiber optics that reach a splitter and then separate and reach each home. In this kind of access network data transference to and from houses happen using an ONT (Optical Network Terminator) that's connected to the fiber optic for home which itself then reaches an Optical Splitter and then becomes the main trunk of fiber optics

which itself reaches an Optical Line Terminator (OLT) in the teleco's CO, the OLT converts the optical signals into electrical ones and gives it to the edge router.

- 4- Dial Up: The dial up provider provides a phone number by calling which your Phone line will be used in an UNSIMILAR and "UN-Optimized" fashion from DSL, to send the digital data.

One Main topic to ask is: Why don't

Files

Are sequences of bytes holding values. Based on the complexity of reading them they are grouped under two generic categories:

- Text Files: The bytes will be read thru and based on an encoding (UTF-8,ASCII,etc) as representatives of characters. They could have file formats but it's not really necessary as they rely on a single encoding to do it for them.
- Binary files: The bytes COULD be text or literal values. Some parts MIGHT need to be read based on Encoding some others might be literal values meant to be read as numbers! They RELY on format to address this.

Binary Files

Binary executables machine code on mainstream OSs rely on an unobfuscatable basic skeleton that NEEDs to have the sections .Code,.Data and etc in one form or another while the symbols could be and usually are obfuscated.

This basic structure and the limited number of instructions in Assembly are why Disassembly seems a lot easier and is usually done by Dump apps (such as dumpbin/objdump). All these disassemblers are basically doing is:

- 1-Recognizing the opcodes (instructions that tell CPU to do stuff) from the values within the machine instructions.
- 2-Replacing the opcodes with their equal-in-context mnemonics in assembly.
- 3-Changing the format to that of Assembly instruction (such as AT&T or MASM).

In the case of PE files they may or may not contain machine language, the doctrine for file formats defines the section there must be not the type of data, the way they are represented may or may not allow for much flexibility.

Constants

MessageBox Constants

Beginning with the prefix MB these constants which are used to set flags for MessageBox to change it's appearance are grouped in three self explanatory categories below for your comfort:

- 1- Button templates – Define the buttons that your Message box will have:

MB_OK 0x00000000L

MB_OKCANCEL 0x00000001L

MB_ABORTRETRYIGNORE 0x00000002L

MB_YESNOCANCEL 0x00000003L

MB_YESNO 0x00000004L

MB_RETRYCANCEL 0x00000005L

- 2- Default Button Setter – Define which button will be the default one **from left to right**

(The one that is selected by default):

MB_DEFBUTTON1 0x00000000L

MB_DEFBUTTON2 0x00000100L

MB_DEFBUTTON3 0x00000200L

MB_DEFBUTTON4 0x00000300L

3-Icon setters – Define the Icon within MessageBox:

MB_ICONHAND 0x00000010L

MB_ICONQUESTION 0x00000020L

MB_ICONEXCLAMATION 0x00000030L

MB_ICONASTERISK 0x00000040L

these icon constants do have alternate names as well:

MB_ICONWARNING *MB_ICONEXCLAMATION*

MB_ICONERROR *MB_ICONHAND*

MB_ICONINFORMATION *MB_ICONASTERISK*

MB_ICONSTOP *MB_ICONHAND*

Types

In order to have a level of compiler-implementation independency Microsoft decided to define their own types, instead of direct reliance on compiler types.

HINSTANCE: A pointer sized datatype used to keep track of the handles.

LPSTR / PSTR: A pointer to an array of 8-bit characters, which MAY be terminated by a null character.

Questions to ask AI

Witcher 2 Binary

investigation

It seems while .text section provides close to no clues, as we doom-scroll upward since a certain point we start seeing things like `GetModuleHandleW`, `OpenMutexW`, `CreateMutexW`, `R CloseHandle KERNEL32.dll`, `GetDlgItem`, , `DialogBoxParamW`, `SetWindowTextW`, `wsprintfW`, `SHELL32.dll` and etc.

In another place I can literally see literal piles of C++ code defining a namespace “and `hkHavok%Classes`”

RE3 Remake

Investigation

Upon reaching mscorelib things start to look readable.

execommon.inl

A file loaded in visual studio release configuration while using break points.

Visual Studio Sentience

Visual studio seems to be aware of the location of the C files of a static library built as a separate project of the same solution, the actual lib file of which I include in another project within the same solution IN different folders. This happens when I use VS in release mode with breakpoints. It loads the said C file.

References

- 1- C How to Program by Dietel brothers
- 2- C++ How to Program by Dietel brothers
- 3- Programming Windows by Charles Petzold
- 4- x86 Assembly by Kip Irvine

The list will be added upon

Some of the BEST books the existence of which had I been aware of , I might've reconsidered writing significant portions of this book:

Computer Systems a Programmer's Perspective

Thinking in C++ Bruce Ekel

Subject 3: OpenGL

Flow

OpenGL is implemented within the driver of GPUs as dynamic libraries, the static reference library and function declarations header of which is typically provided as a part of OS's sdk (at least in the case of windows).

Specification

Sets a set of basic functions that are to be provided within the implementation

Windows

The window's function for context creation always creates the context for OpenGL 1.1 for backward compatibility which is why we gotta use

Terms

CSFOA: *Conditional Symbolic Function-Overload Abstraction.*

.*****
***** Generic methods: *****
***** We can limit the typeparameter in class using notion.*****